

Website Copilot

Metadata Filter, Hybrid Search, RAG Retriever Tool

專案進度報告 · 2026.07.22

薛耀智

大綱

1. Metadata Filter

— Metadata 注入・實作成果

2. Hybrid Search

— Qdrant BM25 → Milvus BGE-M3

3. RAG Retriever Tool

— 檢索工具化・Agent 整合

Metadata Filter

核心問題

不同類型的頁面共用同一向量空間，語義重疊導致**跨類別幻覺**。

問題	範例	影響
公告當論文	「賀！某某論文獲獎」混入論文查詢	LLM 誤以為是正式論文
跨年混淆	歷年活動與最新消息混合	時間範圍查詢失準
名稱誤認	成員姓名出現在不同脈絡	歸屬判斷錯誤

Metadata 注入

- 爬蟲 URL 解析：
 - `/news` → `announcement`
 - `/publication` → `paper`
 - `/members|/advisor` → `personnel`
- 網頁資料結構：
 - 重點內容（網址/Markdown/圖片）
 - `metadata`（內容屬性）
 - `crawl_info`（爬蟲環境）

實作成果

- **Metadata filter 作為工具參數**：Agent 可依查詢意圖動態選擇過濾條件（如 `page_type="paper"`）
- **Paper 節點分數整體偏低（0.37–0.38）**：向量相似度不足導致 Dense Search 設定的 `cutoff=0.4` 排除所有資料，須依賴 Hybrid Search 平衡

Hybrid Search

核心問題

- Paper 節點分數整體偏低（0.37–0.38），Dense Search 會排除所有資料
- 語義不同但關鍵字相近的內容無法被召回

Hybrid Search

同時以 Dense Vector 與 Sparse Vector 檢索，再將兩者分數融合排序

Vector 類型	說明	模型
Dense	語義壓縮，同義詞與上下文理解	OpenAI Embeddings, BGE
Sparse	關鍵字比對，特定術語精準匹配	BM25, BGE-M3

Vector Store With Hybrid Search

Vector Store	Qdrant (BM25)	Milvus (BGE-M3)
Sparse model	Qdrant/bm25 (純 BM25)	BAAI/bge-m3 (神經稀疏編碼)
中文支援	✗ 不支援	✓ 原生支援
融合演算法	Weighted: $\alpha \times \text{dense} + (1-\alpha) \times \text{sparse}$	RRF: $1/(k+\text{rank}_d) + 1/(k+\text{rank}_s) \rightarrow$ Weighted: $w_d \times \text{dense} + w_s \times \text{sparse}$

最終選擇：Milvus + WeightedRanker

Milvus Vector Store

RRFRanker (預設)

$$\text{score} = 1/(k + \text{rank_dense}) + 1/(k + \text{rank_sparse})$$

- 只看排名、不比原始分數 → sparse 干擾過大
- **實驗室成員問題測試**：成員覆蓋僅 **7 位**

WeightedRanker (最終選擇)

$$\text{score} = w_{\text{dense}} \times \text{dense_score} + w_{\text{sparse}} \times \text{sparse_score}$$

- 加權計算 Dense 和 Sparse 原始分數
- **實驗室成員問題測試**：成員覆蓋 **12 位** (與 Qdrant BM25 持平)

Hybrid Search + Metadata Filter

Q5 查詢：「實驗室近三年發表過哪些論文？」

無 filter → Top-10 混入 personnel pages (Score 0.95–0.98)
論文頁面 (Score 0.62–0.63) 被淹沒
Faithfulness 0% · Relevancy 0%

加 filter → filter_dict={"page_type": "paper"}
Top-10 100% 為論文頁面
Faithfulness 100% · Relevancy 有改善空間

RAG Retriever Tool

檢索方法與工具封裝

- RAG retriever tool 參數：
 - `filter_dict`：篩選資料的 metadata 條件（如 `{"page_type": "paper"}`）
 - `similarity_top_k`：設定檢索結果回傳數量上限
- **StructuredTool 封裝**：採用 Pydantic schema 定義輸入（`query`、`filter_dict`、`similarity_top_k`），回傳純文字格式化結果供 Agent 直接讀取

LangGraph Agent 整合

- `create_webpage_retriever_tool()` 為高層工廠函數，傳入設定檔名稱即可獲得建置好的 `StructuredTool`
- **Agent 工具調用**：將工具傳入 `create_agent(model, [tool])`，LLM 在推理過程中自主決定何時呼叫工具並填入參數（如 `query`、`filter_dict`）
- **工具資源釋放**：Agent 結束工作後調用 `tool.rag.close()` 釋放向量儲存資源

總結

1. **Metadata Filter** — 從爬蟲 URL 解析出 `page_type`，注入 LlamaIndex 節點 metadata，在向量檢索前預先隔離跨類別雜訊
2. **Hybrid Search** — 導入 Milvus + BGE-M3 稠密稀疏雙軌檢索，以 WeightedRanker 融合分數，補回低語義相似度但高關鍵字匹配的節點
3. **Retriever Tool** — 將檢索能力包裝為 LangChain `StructuredTool`，使下游 Agent 可動態選擇過濾條件與檢索參數